Introduction to Databases, 2e-B.Sc.
1ST semester 2019–2020, University of Antwerp
Prof. Dr. Toon Calders and M.Sc.Len Feremans

Universiteit Antwerpen

Exam Introduction to Databases
14 January 2019

- The final exam contains 5 problems and is scheduled for 4 hours.
- Please verify if your question booklet consists of this cover page, 5 pages of questions, and 7 pages with supplementary lecture slides that are printed legibly. If this is not the case, contact your tutor immediately.
- The exam questions address both the theory and exercise parts of the lecture. For questions marked with an asterisk (*), *supplementary lecture slides* are provided at the end of this booklet.
- No electronic devices (such as notebooks, smartphones or smartwatches) are allowed.
- Usage of course material is not allowed.
- *Insufficiently detailed* answers are *void* (e.g.: you can't just answer "yes" or "no").
- Make sure to clearly mark your final answer to the questions; that is: do not only show the derivation, but make sure to state your conclusions in an unambiguous way.
- Take pride in your work: answer clearly and hand in a readable exam copy!

- For Problem 1 and 2 you can complete the SQL assignment using the available computers, which include postgres and DataGrip. The exam supervisor will collect solutions. All other questions must be answered on paper.

Problem 1: Entity Relational Model and Relational Design (5 points)

Question 1.2 and 1.3 may be answered using the class computer (paper is also possible).

Your task is to model a game inspired by Dungeon and Dragons. Carefully read the following requirement analysis.

A game consist of 1 or more levels. Each level has a unique name and consist of a *m x n* grid of cells. Each cell has grid coordinates (e.g. <0,1>) and is uniquely identified in the game by the combination of the level name and the grid coordinates. Additionally, a cell is also uniquely identified by a unique code. A cell has description (e.g. "The wizard office of Gandalf" or "The eye of Sauron fire") and contains zero or more items and characters. There are two types of cells: A terrain cell and a dungeon cell. A terrain cell contains a description of the environment (e.g. "It was a cold winter night."). A dungeon cell is connected to other dungeon cells that are accessible from that cell (e.g. From dungeon cell (1,2) you can visit cell (1,1), but not (1,3) because there's a wall). A character has a unique name (e.g. "Bilbo Baggins"), health points and attack and defence points, and zero or more weapons. An item has a unique code, description and is either a Potion or a Weapon. A potion has health points. A weapon has attack and defence points.

1. Create an Entity-Relationship diagram that models as faithfully as possible the above description. Clearly indicate entities, relationships, primary keys, cardinalities and participation constraints.
2. (*) Create the necessary create table statements to implement this ER-diagram as a relational database. You can start from the incomplete definitions in figure 1. Define all necessary schema-level constraints, such as primary keys, foreign keys and domain constraints. Active database instructions such as "On delete/update cascade/..." are not required.
3. Create a check to assert that for each dungeon cell that is connected to another dungeon cell, the connected cells are also spatially connected, that is, they are next to each other in the grid. (for instance cell (3,4) is spatially connected to (2,4), (4,4), (3,3), and (3,5). TODO

```
create table Game(
   id int
);

create table Level(
   name   varchar,
   game_id int
);

create table Cell(
   code int,
   description varchar,
   level_name varchar
);
```

```
create table DungeonCell(
   code int
);

create table Character(
   name varchar,
   healtpoints int,
   attackpoints int,
   defencepoints int
);
```

Figure 1. Start of solution for create table statements.

Problem 2: SQL and Relational Algebra (5 points)

Question 2.2. may be answered using the class computer (paper is also possible). A minimal example schema will be made available.

The database schema for an internet movie database website is shown in Figure 2.

Actor:

act_id	act_fname	act_lname	act_gender
5	Tom	Hanks	M
9	Robin	Wright	F
11	Matthew	McConaughey	M
13	Emily	Blunt	F

Genre:

genre_id
Drama
Comedy
Action
Horror

Director:

dir_id	dir_fname	dir_lname
28	Robert	Zemeckis
31	Christopher	Nolan
33	Rob	Marshall
39	John	DeLuca

Movie_genres:

mov_id	genre_id
1	Drama
1	Comedy
7	Action
7	Sci-fi

Movie:

mov_id	mov_title	mov_year	mov_dur
1	Forest Gump	1994	2h22
7	Interstellar	2014	2h49
9	Mary Poppins Returns	2018	2h11
11	Green Book	2018	2h10

Movie_director:

mov_id	dir_id
1	28
7	31
9	33
11	67

Rating:

mov_id	rev_id	stars	comments
1	101	5	... A masterpiece!
1	103	3.5	... predictable.
7	103	5	... Super!
11	107	4.5	Greatful to have seen this

Reviewer:

rev_id	rev_name
101	Rachel Tsang
103	John Hendrickx
104	TheTopDawgCritic
107	Lotus-3

Movie_cast:

mov_id	act_id	Role
1	5	Forest Gump
1	9	Jenny
7	11	Joseph Cooper
9	13	Mary Poppins

Figure 2. Schema for internet movie database.

(Problem 2 – continued)

Write the following queries in SQL:

a. Return the firstname and lastname of all actors who played together with Tom Hanks in a movie
b. List all the Drama movies with title, year, name of the director and average star rating ordered by year
c. Report each genre of movies together with the number of movies and their average star rating
d. Find all pairs of movie genres that have never been assigned to the same movie (e.g. "Horror" and "Family").

(*) Write the following queries in Relational Algebra:

e. List the first and last names of all the actors who were cast in the movie Forest Gump and the roles they played in that production.
f. Find the name of all directors (first and last names) who directed a movie that casted the role "Forest Gump."
g. List all the actors who have not acted in any movie between 1990 and 2000

Problem 3: Database design (4 points)

Consider the following database relation that stores data on football matches. Each row in the table is related to a player that participated in a match. The date of the match is stored, as well as the name of the competition, the season of the competition, the identifier of the match, the team of the player, the player's registration number, and his shirt number during that game. Furthermore, if the player received a yellow or a red card during the game, this is recorded in the attributes Y and R (x indicates a card was given, - that no card was given). If a player receives multiple cards during a game, the tuple is duplicated for each card. Furthermore, a remark can be stored. This remark can be related to a card a player receives (giving the reason), or a general remark such as that the player got injured and had to leave the pitch.

Below a table is **given** to illustrate the principle; on 26/1/20 there was a match between RAFC and ZW in competition JPL. The ID of this match is 001. For RAFC player Ze played in this match with shirt number 7. He received a yellow card for "inappropriate goal celebration", and a second yellow card resulting in a red card for "discussion". In the same game, player Bossut with shirt number 1 played for ZW with the remark that he got injured in minute 67. Also player Govea played for ZW in that game, with shirt number 8 and without remarks On 22/1/20 there was a game in competition CC between CB and ZW. For CB, Vormer played with number 25. He received a yellow card and got injured in minute 38. In the same game Govea, playing for ZW received a direct red card for a faul.

Date	Com	Season	MID	Team	Player_ reg_nr	Player_ name	Shirt _Nr	Y	R	Remark
26/1/20	JPL	19-20	001	RAFC	12345	Ze	7	x	-	Inappropriate goal celebration
26/1/20	JPL	19-20	001	RAFC	12345	Ze	7	x	x	Discussion
26/1/20	JPL	19-20	001	ZW	23456	Bossut	1	-	-	Injury in min. 67
26/1/20	JPL	19-20	001	ZW	45678	Govea	8	-	-	-
22/1/20	CC	19-20	010	CB	89012	Vormer	25	x	-	Discussion
22/1/20	CC	19-20	010	CB	89012	Vormer	25	-	-	Injury in min. 38
22/1/20	CC	19-20	010	ZW	45678	Govea	9	-	x	Faul

In this relation, the following rules hold:

1. The **date** of a match and the competition it is played in, determine the season.
2. The **player** registration number is unique for each player. Player names are stored consistently.
3. Within one season of a competition there cannot be two different matches with the same match identifier (MID).
4. A player cannot change his shirt number during a match. It can be different between matches.
5. A player cannot change teams within one season of the same competition. However, a player can play for different teams in different competitions during the same season. (For example: "Ze" plays for "RAFC" in "JPL" and might play for team "Cameroon" in competition "AC")

Question:

a. (*) For each of the rules 1-5, if possible, state it *as precisely as possible* as a functional or multi-valued dependency. If it is not possible to represent it exactly, explain *why* this is not possible.
b. List all candidate keys in this relation.
c. (*) Give a lossless decomposition of the relation in Boyce-Codd Normal Form.
d. (*) Is this decomposition dependency-preserving? Give arguments to support your answer!

Problem 4: Concurrency (4 points)

Consider the following two sequences of actions, listed in the order they were submitted to the database. Timestamp of T1 < timestamp of T2 < timestamp of T3.

T1: R(X), T2: W(X), T2: W(Y), T3: W(Y), T2: Commit, T1: W(Y), T1: Commit, T3: Commit

T1: R(X), T2: W(Z), T1: R(Z), T2: R(Y), T3: R(X), T2: W(Y), T2: Commit, T1: W(X), T3: W(Y), T1: Commit, T3: Commit

For both sequences answer the following questions:

a) Does the given sequence represent a conflict-serializable schedule?
b) Under which of the following protocols can the transactions be executed in exactly this order?
c) If the transactions cannot be executed in this order, which transaction(s) will be aborted and rolled back?
d) If transactions are aborted, is the schedule recoverable?

- Timestamp-based protocol without Thomas' write rule
- Timestamp-based protocol with Thomas' write rule
- 2-Phase Locking with wait-die deadlock prevention

Explain your answer. Answers without explanation will be considered void.

Problem 5: NoSQL (2 points)

In a *partition-tolerant distributed data management system*, the system will continue serving read and write requests, even in case of a network partitioning (for instance: unresponsive node). Updates will be asynchronously distributed throughout the system and, if properly implemented, the data stored will eventually be consistent. To realize this *eventual consistency* we need a mechanism to recognize and reconcile incompatible versions. Explain what vector clocks are and how they can be used in this context.

Supplementary material:

Defining Foreign Keys

```
CREATE TABLE Orders
(
    O_Id int NOT NULL,
    OrderNo int NOT NULL,
    P_Id int,
    PRIMARY KEY (O_Id),
    FOREIGN KEY (P_Id) REFERENCES Persons(P_Id)
)
```

Does not have to be the same attribute name. But number and types need to match

Could also be multiple attributes

Enforces that every Orders.P_Id occurs in the Persons table in the column P_Id.

Assertions

(not supported in Oracle/Postgres, uses CREATE TRIGGER instead)

MovieExec(name, address, certN, netWorth)
Studio(name, address, presCertN)

- No one may be the president of a studio, unless this person has a net worth of at least $1M :

```
CREATE ASSERTION RichPres CHECK
    (NOT EXISTS (
        SELECT * FROM MovieExec, Studio
        WHERE certN = presCertN AND netWorth < 1000000));
```

- Every executive name has only one address :

```
CREATE ASSERTION FunctionalDependency CHECK
    (NOT EXISTS (
        SELECT * FROM MovieExec mv1, MovieExec mv2
        WHERE mv1.name = mv2.name AND mv1.address <> mv2.address));
```

Triggers in SQL

General syntax:

```
CREATE TRIGGER <trigger_name>
 {BEFORE|AFTER}  {{INSERT|DELETE|UPDATE} [OF <row_list>]}* ON <table_name>
 [REFERENCING  [NEW ROW|TABLE AS <new_name>] [OLD ROW|TABLE AS <old_name>]]
 [FOR EACH ROW|STATEMENT [WHEN (<trigger_condition>)]]
BEGIN
  <trigger_body>
END;
```

Universiteit Antwerpen

* SQL'99 allows both ROW and TABLE/STATEMENT level triggers. Oracle allows only ROW level, hence the keyword ROW is omitted.

Examples

- Likes(Drinker, Beer)
- Serves(Pub, Beer)
- Visits(Drinker, Pub)

Give all drinkers who do not like "Guiness"

$$(\pi_{Drinker}\ Likes \cup \pi_{Drinker}\ Visits) - \pi_{Drinker}\ \sigma_{Beer="Guiness"}\ Likes$$

All drinkers

Drinkers who like Guiness

Universiteit Antwerpen

Examples

- Likes(Drinker, Beer)
- Serves(Pub, Beer)
- Visits(Drinker, Pub)

Give all drinkers who visit a bar that serves "Guiness"

$$\pi_{Drinker}\, \sigma_{GPub=Pub}\, (Visits \times (\rho_{GPub/Pub}\, \sigma_{Beer="Guiness"}\, Serves))$$

Give all drinkers who visit a bar that serves a beer they like

$$\pi_{Drinker}\, \sigma_{VP=SP \wedge SB=Beer \wedge Drinker=VD}\, (Likes \times \rho_{SP/Pub,\, SB/Beer}\, Serves \times \rho_{VD/Drinker,\, VP/Pub}\, Visits)$$

=

$$\pi_{Drinker}\, \sigma_{SB=Beer \wedge Drinker=VD}\, [Likes \times \sigma_{VP=SP}\, (\rho_{SP/Pub,\, SB/Beer}\, Serves \times \rho_{VD/Drinker,\, VP/Pub}\, Visits)]$$

Keys & Functional Dependencies

A set of one or more attributes $\{A_1, A_2, ..., A_n\}$ is called a **key** for a relation *R* if:

(1) $\{A_1, A_2, ..., A_n\}$ functionally determines *all attributes* of *R*.

(2) *No proper subset* of $\{A_1, A_2, ..., A_n\}$ functionally determines all attributes of *R*.

→ In other words, a key must be a *minimal set of attributes* for which property (1) holds.

A set of attributes $\{A_1, A_2, ..., A_n\}$ that contains a key of relation *R* is called a **superkey** of *R*.

→ Every key is also a superkey.

A Complete Set of Inference Rules

Armstrong's Axioms:

The following three axioms allow for deriving any FD that follow from a given set of FD's.

1. Reflexivity:
   If $\{B_1, B_2, ..., B_m\} \subseteq \{A_1, A_2, ..., A_n\}$,
   then $A_1, A_2, ..., A_n \rightarrow B_1, B_2, ..., B_m$ (→ *"trivial FDs"*)
2. Augmentation:
   If $A_1, A_2, ..., A_n \rightarrow B_1, B_2, ..., B_m$,
   then $A_1, A_2, ..., A_n, C_1, C_2, ..., C_k \rightarrow B_1, B_2, ..., B_m, C_1, C_2, ..., C_k$,
   for any set of attributes $C_1, C_2, ..., C_k$. (→ *"non-trivial FDs"*)
3. Transitivity:
   If $A_1, A_2, ..., A_n \rightarrow B_1, B_2, ..., B_m$ and $B_1, B_2, ..., B_m \rightarrow C_1, C_2, ..., C_k$,
   then $A_1, A_2, ..., A_n \rightarrow C_1, C_2, ..., C_k$. (→ *"closure algorithm"*)

Universiteit Antwerpen

Normal Forms of Relations

In the following, let X be a *set of attributes* and Y be a *single attribute* of a relation R, and let F be a given set of FD's on R.

R is in **Third Normal Form** (3NF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is a superkey of R or Y is an attribute of a key of R.

R is in **Boyce-Codd Normal Form** (BCNF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is a superkey of R.

Universiteit Antwerpen

Algorithm for Computing a Decomposition in BCNF (not dependency-preserving!)

Given a relation R with attributes A, a set of keys K,
and a set of FD's F.

If there is a non-trivial FD $X \rightarrow Y$, where X is not a superkey of R,
then decompose R into:

- $S = \pi_{X^+(F)}(R)$ with the new key X;
- $T = \pi_{X \cup (A - X^+(F))}(R)$ by taking over all remaining keys from R.

Then recursively decompose S and T into BCNF using the
remaining (non-violated) FD's in F.

Universiteit Antwerpen

Algorithm for Computing a Dependency-Preserving Decomposition in 3NF

Given a relation R with attributes A, a set of keys K,
and a *minimal basis* of FD's F.

Decompose R into:

- $R_1 = \pi_{K'}(R)$ where K' is a key in K which becomes key of R_1;
  (if there are multiple keys in R, choose one)
- for each $X \rightarrow Y$ in F,
  the projection $R_{i+1} = \pi_{X \cup Y}(R)$ forms a new relation
  in the decomposition with key X.

Universiteit Antwerpen

Multivalued Dependency

A multivalued dependency (MVD), denoted as

$A_1,A_2,...,A_n \rightarrow\rightarrow B_1,B_2,...,B_m$

holds for a relation R if, for each pair of tuples t, u of R, which agree on all the A's, there is a tuple v in R that also agrees

- with both t and u on the A's;
- with t on the B's;
- with u on *all other attributes* of R not among the A's and B's.

Example:

	starName	→→	city,street		title,year
	A's		B's		others
t	a_1		b_1		c_1
	Ford		Malibu, Maple Str.		Star Wars, 1977
v	a_1		b_1		c_2
	Ford		Malibu, Maple Str.		Star Wars, 1980
u	a_1		b_2		c_2
	Ford		Hollywood, Mulh. Dr.		Star Wars, 1980

Universiteit Antwerpen

The Two-Phase Locking Protocol

- This protocol ensures conflict-serializable schedules.
- Phase 1: Growing Phase
  - Transaction may obtain locks
  - Transaction may not release locks
- Phase 2: Shrinking Phase
  - Transaction may release locks
  - Transaction may not obtain locks
- The protocol assures serializability. It can be proved that the transactions can be serialized in the order of their **lock points** (i.e., the point where a transaction acquired its final lock).

Universiteit Antwerpen